Student Platform - Technical Specification

KickstartSkills Student Dashboard

1. Authentication & Registration System

1. Institution Code Validation

What it is: This is the very first step to make sure you belong to an institution (like a college or university) that KickstartSkills is partnered with.

Input Field(s):

- **Institution Code:**
- **Type:** Text/Alphanumeric
- A short code (like "ABC123") provided by your college/university.
    - **Why we need it:** To instantly check if your institution is recognized by KickstartSkills. This prevents unauthorized sign-ups and ensures you get access to the right resources for your college.

---

2. Registration Form (Multi-step)

This is where you start filling in your core details. Since it's "multi-step," it means you won't see everything at once, making it less overwhelming.

Input Field(s) - Examples:

- **Email Address:** Your primary email (e.g., student@example.com)
- **Password:** A secret combination of letters, numbers, and symbols you create.
- **Confirm Password:** Re-enter your chosen password.
- **First Name:** Your given name (e.g., "Priya")
- **Last Name:** Your family name (e.g., "Sharma")
- **Mobile Number:** Your phone number (e.g., +91 9876543210)

---

3. Profile Setup

Once you've done the basic registration, you'll fill in more details to complete your profile.

Input Field(s) - Examples:

- **Date of Birth:** Your birthdate (e.g., 2005-01-15)
- **Gender:** (e.g., Male, Female, Non-binary, Prefer not to say)
- **Current Address (Line 1, Line 2, City, State, Pincode):** Your physical residence.

- **Educational Background:**
  - **Highest Qualification:** (e.g., "12th Pass", "B.Tech", "M.Sc.")
  - **Degree/Program Name:** (e.g., "Computer Science Engineering", "B.A. Economics")
  - **University/College Name:** (e.g., "Nagpur University", "XYZ Institute of Technology")
  - **Year of Graduation (Expected/Actual):** (e.g., "2026", "2024")

- **Technical Interests:**
  - **Checkboxes/Multi-select List:** (e.g., "Web Development", "Data Science", "Cybersecurity", "AI/ML", "Cloud Computing", "Mobile App Development", etc.)

---

4. Document Upload

This is where you'll provide digital copies of important documents.

Input Field(s):

- **Profile Picture:** A clear photo of yourself.
- **Academic Certificates:** Copies of your mark sheets or degree certificates.
- **ID Proofs:** Copy of a government-issued ID (e.g., Aadhar Card, Passport, Driving License).

---

5. Payment Integration

If there's a fee for registration or an exam, this is where you'd handle it.

Input Field(s) (Handled by Payment Gateway):

- **Credit/Debit Card Details:** Card number, expiry date, CVV.
- **UPI ID:** For UPI payments.
- **Net Banking Details:** Bank selection, username/password (entered on bank's secure page).
  - (Note: These details are typically handled by the secure payment gateway (Razorpay/Stripe) and are not stored directly by KickstartSkills for security reasons.)

---

6. Email Verification

Process:

- After you submit your registration, the system automatically sends an email to the address you provided.
- This email contains a unique link or a code.
- You click the link or enter the code on the KickstartSkills website.
    - **Why we need it:** To confirm that the email address you provided is real and that you own it. This is a crucial security measure to prevent fake accounts and ensure you receive important communications.

---

7. Password Management

This covers setting up and changing your password securely.

Input Field(s):

- **New Password:** When creating or resetting.
- **Confirm New Password:** To verify the new password.
- **Old Password:** When changing an existing password.
- **Email Address (for forgotten password):** To initiate a password reset.
- **OTP/Verification Code (for forgotten password):** Sent to your email/phone to confirm identity.
    - **Why we need it:** To allow you to securely create, update, and regain access to your account if you forget your password.

2.StudentPlatform - KickstartSkills: Data Collection for Dashboard Features

1. Personal Profile Summary

This section needs core student identification and contact details.

- **Input Fields:**
    - **Student ID:** (Text, unique identifier)
    - **First Name:** (Text)
    - **Last Name:** (Text)
    - **Email Address:** (Email format)
    - **Date of Birth:** (Date picker)

    - **Gender:** (Dropdown: Male, Female, Non-binary, Prefer not to say)
    - **Phone Number:** (Text, numerical with country code)
    - **Address:** (Text area: Street, City, State/Province, Postal Code, Country)
    - **Program/Course Enrolled:** (Dropdown/Text: e.g., "B.Tech Computer Science", "MBA Finance")
    - **Current Year/Semester:** (Dropdown/Number: e.g., "3rd Year", "Semester 5")

2. Progress Tracking Widget

To show progress in internship phases, we need to know the student's current status and milestones.

- **Input Fields (often updated by system or administrator):**
    - **Internship ID:** (Text, unique identifier for the internship)
    - **Internship Program Name:** (Text)
    - **Current Phase:** (Dropdown: e.g., "Application", "Interview", "Onboarding", "Phase 1: Learning", "Phase 2: Project Work", "Completion")
    - **Phase Completion Dates:** (Date picker for each phase, likely updated by system/mentor)
    - **Mentor/Supervisor Name:** (Text, if applicable to the internship)

3. Upcoming Tasks

This requires details about assignments and deadlines.

- **Input Fields:**
    - **Task Name/Title:** (Text)
    - **Task Description:** (Text area)
    - **Due Date:** (Date picker)
    - **Due Time:** (Time picker)
    - **Assigned By:** (Text, e.g., "Instructor Name", "System")
    - **Associated Course/Module:** (Dropdown/Text)
    - **Task Type:** (Dropdown: e.g., "Assignment", "Quiz", "Project Milestone", "Reading")
    - **Priority Level:** (Dropdown: Low, Medium, High)

4. Recent Activity Feed

This is primarily *output* generated by the system based on user actions or system updates. However, some actions might originate from input.

- **Input Fields:**

    - **Submission of an Assignment:** (System-generated upon user action)
    - **Completion of a Quiz:** (System-generated upon user action)
    - **Profile Update:** (System-generated after user saves profile form)
    - **New Notification Posted:** (Admin input for broad announcements)

5. Quick Actions Panel

This isn't about collecting data *for* the panel, but about the *actions* themselves, which might lead to data collection (e.g., "Submit Assignment" leads to an assignment submission form).

- **Input Field (example of an action that uses an input field):**
    - **Feedback/Support Request:** (Text area for message, Subject line text)

6. Notification Center

In-app notifications and alerts. Similar to the activity feed, these are system-generated messages.

Data Collected (System-Generated):

- **Notification Type:** (e.g., "New Assignment," "Grade Posted," "Important Announcement")
- **Notification Message:** Text
- **Timestamp:** Date/Time
- **Read Status:** Boolean (True/False)

7. Performance Metrics

This data is usually derived from other inputs (like task completion or exam scores).

- **Input Fields (examples of data sources for metrics):**
    - **Exam/Quiz Score:** (Number, collected via a quiz/exam submission form or manually entered by instructor)
    - **Task Status:** (Dropdown: "Pending", "In Progress", "Completed", "Overdue" - updated as tasks are managed)
    - **Assignment Grade:** (Number/Letter Grade, entered by instructor/grader)
    - **Attendance Record:** (Checkbox/Dropdown: Present, Absent, Excused - collected via attendance system or manual entry)

3.Examination & Assessment System: Input Fields

1. Exam Registration: Course selection and scheduling interface

This is where a student tells the system which exam they want to take and when.

- **Input Fields (What the student provides):**
  - **Student ID/Login Information:** To identify the student. (Often pre-filled after login)
  - **Course Name/ID:** Which course's exam are they registering for? (e.g., "Introduction to Python Programming," "Web Development Fundamentals")
  - **Exam Type:** Is it a final exam, a mid-term, a practice test?
  - **Preferred Exam Date/Time Slot:** If the system offers flexibility, the student would pick from available slots.
  - **Confirmation Checkbox:** "I confirm my registration for this exam."
- **System Output/Display (What the system shows the student):**
  - Available exam dates and times.
  - Confirmation of registration details.
  - Exam rules and instructions.

2. Online Test Interface: Timer-based examination system with anti-cheating measures

This is the actual exam itself. The "inputs" here are the student's answers.

- **Input Fields (What the student provides):**
  - **Answers to Questions:** This is the core input. Depending on the question type (see below), it could be:
    - Selected option for Multiple Choice.
    - Code entered in a code editor for Coding Questions.
    - Text typed into a text box for Essay Type questions.
  - **Confirmation/Submit Button:** To finalize and submit the exam.
- **System Output/Display (What the system shows the student):**
  - The exam questions.
  - Remaining time on the timer.
  - Current question number/progress.
  - Anti-cheating alerts (if any suspicious activity is detected).

3. Question Types Support: Multiple choice, coding questions, essay type

This defines the *format* of the answers collected in the Online Test Interface.

- **Data Collected (from student):**
  - **Multiple Choice:** The ID or value of the selected option (e.g., 'A', 'B', 'C', 'D').

- **Coding Questions:** The actual code written by the student (as a string/text).
- **Essay Type:** The text of the essay written by the student (as a long string/text).

4. Auto-save Functionality: Automatic saving of answers during exam

- **Data Collected (from student, automatically by system):**
  - **Periodic Snapshots of Answers:** The system continuously saves the student's current answers to each question at regular intervals. This means if the student's connection drops, their progress is not lost.
  - **Timestamp:** Records when the answers were saved.

5. Result Display: Immediate results with detailed score breakdown

- **System Inputs:**
  - **Student's Submitted Answers:** Used for grading.
  - **Correct Answers/Grading Rubrics:** Used to compare against student answers.

6. AI Recommendations: Skill area suggestions based on performance

- **System Inputs:**
  - **Student's Exam Scores and Performance Data:** The AI analyzes this information.
  - **Pre-defined Skill Areas:** The system has a list of skills it can recommend.

7. Re-examination Portal: Interface for retaking failed exams

- **Purpose:** Allows students to re-attempt exams.
- **Likely Input Fields:**
  - **Student ID/Email:** To identify the student.
  - **Exam Name/ID (Retake):** A dropdown list of exams the student is eligible to retake.
  - **Reason for Retake (optional):** A small text area.
  - **Confirmation Checkbox:** "I understand the re-examination policies."
  - **Register Button:** To initiate the re-registration process.

---

7. Score History: Complete examination history with analytics

- **Purpose:** Provides an overview of a student's past exam performance.
- **Input Fields:** None, primarily a display and filtering interface.
- **Likely Filtering/Viewing Options (not input fields for data, but for navigating data):**
  - **Filter by Course:** Dropdown to show history for a specific course.
  - **Filter by Date Range:** Date pickers to narrow down results.
  - **Search Bar:** To find specific exams.
- **Displayed Information:**
  - Table of past exams with scores, dates, and status (Pass/Fail).
  - Graphs or charts showing performance trends over time.

4. Course & Learning Management System

1. Course Catalog

This is how courses are displayed and found.

- **Administrator/Content Creator Inputs:**
  - **Course Title:** (e.g., "Introduction to React.js") - *Text Field*
  - **Course Description:** (e.g., "Learn the fundamentals of React.js...") - *Large Text Area*
  - **Course Thumbnail/Cover Image:** (e.g., Image of a React logo) - *File Upload*
  - **Course Category/Tags:** (e.g., "Web Development", "Frontend") - *Multi-select/Tag Input*
  - **Instructor Name(s):** (e.g., "John Doe") - *Text Field*
  - **Course Difficulty Level:** (e.g., "Beginner", "Intermediate", "Advanced") - *Dropdown/Radio Button*
  - **Estimated Course Duration:** (e.g., "10 hours", "4 weeks") - *Text Field/Number Input with Unit*
  - **Prerequisites (Optional):** (e.g., "Basic JavaScript knowledge") - *Text Area/Multi-select from other courses*
  - **Pricing/Access Type:** (e.g., "Free", "Paid", "Subscription") - *Radio Button/Dropdown*
  - **Syllabus/Outline:** (Structured content, possibly nested modules/lessons) - *Rich Text Editor/Structured Data Input*

- **Student Inputs (for filtering/searching):**
  - **Search Query:** (e.g., "React", "Python for Data Science") - *Text Field*
  - **Category Filter:** (e.g., "Web Development", "Marketing") - *Dropdown/Checkbox List*
  - **Difficulty Level Filter:** (e.g., "Beginner", "Advanced") - *Dropdown/Checkbox List*
  - **Instructor Filter:** (e.g., "John Doe") - *Dropdown/Text Field (for autocomplete)*

2. Video Lecture Player

This is about the video content itself and how students interact with it.

- **Administrator/Content Creator Inputs:**
  - **Video File Upload (Source):** (e.g., `lecture1.mp4`) - *File Upload*
  - **Video Title/Lecture Name:** (e.g., "Lecture 1: React Components") - *Text Field*
  - **Lecture Description (Optional):** (e.g., "This lecture covers the basics of React components...") - *Text Area*
  - **Subtitle File Upload (.srt, .vtt):** (e.g., `lecture1_en.vtt`) - *File Upload*
  - **Associated Course/Module:** (Link video to a specific place in the course) - *Dropdown/Selector*
- **Student Inputs (System-Generated/Interaction-based):**
  - **Play/Pause Action:** (Implicit, system records state) - *Button Click*
  - **Speed Control Selection:** (e.g., 0.5x, 1x, 1.5x) - *Dropdown/Radio Button Selection*
  - **Video Quality Selection:** (e.g., 480p, 720p, 1080p) - *Dropdown/Button Selection*
  - **Bookmark Timestamp:** (User defines a specific point in video) - *Timestamp Input (from player interaction)*
  - **Note Content:** (Text of the note) - *Text Area*
  - **Resume Position:** (System stores `video_id` + `timestamp` automatically) - *No direct student input, system tracks*

3. Course Materials

Providing downloadable resources.

- **Administrator/Content Creator Inputs:**
  - **Material File Upload:** (e.g., `react_slides.pdf`, `code_samples.zip`) - *File Upload*
  - **Material Title:** (e.g., "Lecture 1 Slides", "React Code Samples") - *Text Field*
  - **Material Type:** (e.g., "PDF", "Presentation", "Code Samples") - *Dropdown*
  - **Associated Course/Module:** (Link material to a specific place) - *Dropdown/Selector*

4. Progress Tracking

Monitoring student advancement.

- **Student Inputs:**
  - **Completion of Lecture/Module:** (Implicit, system marks based on video watch completion, quiz pass) - *No direct student input, system tracks*

    - **Quiz Attempt/Submission:** (Answers to quiz questions) - *Various input types depending on quiz (Radio, Checkbox, Text Area)*
- **Administrator/Content Creator Inputs (for setting up tracking):**
    - **Module Completion Criteria:** (e.g., "Watch 90% of videos", "Pass quiz with 70%") - *Configuration settings*

5. Quiz Integration

Embedding assessments.

- **Administrator/Content Creator Inputs:**
    - **Quiz Title:** (e.g., "React Basics Quiz") - *Text Field*
    - **Associated Course/Module:** - *Dropdown/Selector*
    - **Quiz Questions:** (e.g., "What is JSX?") - *Text Area*
    - **Question Type:** (e.g., "Multiple Choice", "True/False", "Short Answer") - *Dropdown*
    - **Answer Options (for MCQs):** (e.g., "A. JavaScript XML", "B. JavaScript Extension") - *Text Fields*
    - **Correct Answer(s):** - *Radio Button/Checkbox Selection*
    - **Explanation for Answer (Optional):** - *Text Area*
    - **Passing Score/Percentage:** (e.g., "70%") - *Number Input*
- **Student Inputs:**
    - **Quiz Answers:** (Selecting options, typing text) - *Radio Button, Checkbox, Text Area*

6. Certificate Generation

Awarding completion.

- **Administrator/Content Creator Inputs (for certificate template):**
    - **Certificate Template Design:** (Layout, logos, font styles) - *Design Editor/File Upload (image/PDF template)*
    - **Certificate Title:** (e.g., "Certificate of Completion") - *Text Field*
    - **Fields to Auto-populate:** (e.g., Student Name, Course Name, Date of Completion) - *Placeholder/Variable Definition*
    - **Authorized Signatory Name/Signature:** (e.g., "Director, KickstartSkills") - *Text Field/Image Upload*
- **Student Inputs:**
    - **Student Name (from profile):** (Implicit, usually pulled from user registration) - *No direct input for certificate generation, pulled from profile*

7. Learning Path (Personalized Recommendations)

Suggesting next steps.

- **Administrator/Content Creator Inputs (for recommendation rules/data):**
    - **Course Tags/Keywords:** (Important for matching interests) - *Multi-select/Tag Input (already covered in Course Catalog)*
    - **Course Dependencies/Sequencing:** (e.g., "Course A must be completed before Course B") - *Structured Data Input*
- **Student Inputs (Implicit/Behavioral):**
    - **Completed Courses:** (System tracks) - *No direct input*
    - **Browse History/Interests:** (System tracks which courses were viewed/searched) - *No direct input*
    - **Ratings/Feedback on Courses (Optional, for better recommendations):** (e.g., Star rating, comments) - *Star Rating, Text Area*

5. Task & Assignment Management

1. Task Dashboard (for viewing tasks)

This isn't about input fields for creating a task, but how the system *displays* information based on tasks that have already been created.

- **List View Filters/Sorts:**
    - **Input needed from user (for filtering/sorting):**
        - Task Status (e.g., "All," "Assigned," "Submitted," "Graded," "Overdue") - *Dropdown/Checkbox*
        - Due Date Range (e.g., "Today," "Next 7 Days," "Custom Range") - *Date Picker*
        - Course/Module Name - *Dropdown/Search field*
        - Task Type (e.g., "Individual," "Group Project") - *Dropdown*
        - Search by Task Title/Keyword - *Text Input*

2. Task Details (for creating/editing a task)

This is where the core information about a specific task is defined.

- **Task Title:**
- **Task Description/Requirements:**
- **Associated Course/Module:**
- **Assignment Type:**
- **Total Marks/Points:**
- **Attachments (for mentor to provide to students):**

3. File Submission System (for students submitting files)

This focuses on how students upload their work.

- **File Upload Area:**
- **Optional Comments/Notes for Submission:**

---

4. Version Control (for students and mentors)

This describes how multiple submissions are handled.

- **No direct "input fields" for version control itself.** This is an *automated system feature* that tracks each time a student submits files for a task.
- **Display for Students/Mentors:**
    - **Information Displayed (not input):**
        - Version Number (e.g., "Submission 1," "Submission 2")
        - Timestamp of Submission
        - Files included in that version
        - Option to view/download previous versions.

5. Collaboration Tools (for group tasks)

This section focuses on managing group-based assignments.

- **Team Name (if applicable):**
- **Add Team Members:**
- **Communication Channel (e.g., Chat, Discussion Forum):**

---

6. Deadline Tracking (for mentors creating tasks)

This defines the timing of the task.

- **Due Date & Time:**
- **Grace Period (Optional):**
    - **Necessary Information:** How much extra time (if any) is allowed after the official deadline.
- **Reminder Notification Settings:**
    - **Necessary Information:** When and how students should be reminded about the deadline.

---

7. Submission Status (Automated and displayed)

This is a system-generated status, not something a user inputs directly.

- **Information Displayed (not input):**
  - **Status:** "Not Started," "Assigned," "In Progress," "Submitted," "Late Submission," "Under Review," "Graded."
  - **Submission Date/Time:** When the student submitted the task.

---

8. Feedback Display (for mentors giving feedback and students receiving it)

This covers the grading and feedback process.

- **Mentor Input Fields (for grading):**
  - **Score/Grade:**
  - **Overall Feedback/Comments:**
  - **Rubric-based Feedback (if applicable):**
    - **Input Field Type:** *Checkboxes / Short Text Inputs per rubric criterion*
    - **Necessary Information:** Specific feedback tied to predefined grading criteria.
  - **Annotated Feedback (on submitted files):**
    - **Input Field Type:** *Annotation tools integrated with file viewer (e.g., highlights, comments directly on PDF/document)*
    - **Necessary Information:** Direct, in-context feedback on the student's submitted work.

---

9. Task History (for viewing past tasks)

Similar to the Task Dashboard, this is for displaying archived information.

- **Filters/Sorts for Archived Tasks:**
  - **Input needed from user (for filtering/sorting):**
    - Completion Year/Semester - *Dropdown*
    - Course/Module Name - *Dropdown/Search field*
    - Task Status (e.g., "All Completed," "Graded") - *Dropdown/Checkbox*
    - Search by Task Title/Keyword - *Text Input*

6. Internship Management

Technologies: React.js dashboard, real-time API integration

1. Internship Status: Current phase tracking with timeline visualization

- **What it does:** Shows where you are in your internship journey (e.g., Applied, Interviewing, Offered, Started, Completed) and when things are happening.

Input Fields/Information to Display:

- **Internship Start Date:** (Date Picker) - When the internship officially began.
- **Internship End Date:** (Date Picker) - When the internship is expected to finish.
- **Current Phase:** (Pre-defined options: Application, Interview, Offer, Onboarding, Active, Completed) - What stage the student is currently in.
- **Next Milestone:** (Text/Date) - What's the next big step or deadline.
- **Timeline Visualization Data:** (System-generated from start/end dates and current phase) - Visual representation of progress.

2. Document Management: Upload and manage internship-related documents

What's needed? For students to easily upload and access important internship documents.

Input Fields/Information to Display:

3. Offer Letter: Digital offer letter viewing and acceptance

- **What the student sees/does:** They'll see their official offer letter and have the option to accept or decline it.
- **Input Fields / Key Information Needed:**
  - **Offer Letter Document:** (System-generated PDF/viewable document)
  - **Accept Button:** (Confirms acceptance)
  - **Decline Button:** (Declines the offer)
  - **Signature/E-signature Field:** (If digital signature is required for acceptance)
  - **Date of Acceptance/Rejection:** (System-recorded)
  - **Optional: Reason for Rejection:** (Text field if they decline)

---

4. Onboarding Checklist: Interactive checklist for joining formalities

- **What the student sees/does:** A list of tasks they need to complete before or at the start of their internship.
- **Input Fields / Key Information Needed:**
  - **Checkbox for each item:** (Student checks when completed, e.g., "Submit ID Proof," "Complete HR Forms," "Set up Email Account")
  - **Completion Date:** (System-recorded when checkbox is marked)
  - **Link to relevant section/document:** (e.g., "Link to HR Portal," "Upload ID Here")

---

5. Project Assignment: View assigned projects with specifications

- **What the student sees/does:** Information about the projects they'll be working on. They usually don't input much here, just view.
- **Input Fields / Key Information Needed (mostly system-provided):**
  - **Project Title:** (Text)
  - **Project Description/Objectives:** (Long text)
  - **Key Deliverables:** (List/text)
  - **Deadline:** (Date)
  - **Supervisor/Mentor:** (Name, Contact Info - pulled from user profiles)
  - **Related Documents:** (Links to project brief, design documents, etc.)

---

6. Team Information: Team member details and contact information

- **What the student sees/does:** Details of their team members. Again, mostly viewing.
- **Input Fields / Key Information Needed (mostly system-provided):**
  - **Team Member Name:** (Text)
  - **Role/Designation:** (Text)
  - **Contact Email:** (Email address)
  - **Contact Phone:** (Optional: Phone number)
  - **Profile Picture:** (Image)

---

7. Daily Check-in: Daily attendance marking with location verification

- **What the student sees/does:** Marks their attendance for the day.
- **Input Fields / Key Information Needed:**
  - **Check-in Button:** (To mark attendance)
  - **Current Location:** (System-detected, requires user permission for GPS)
  - **Timestamp:** (System-recorded)
  - **Optional: Check-out Button:** (If tracking full attendance cycle)

---

8. Timesheet Management: Log daily activities and time spent

- **What the student sees/does:** Records what they worked on and for how long.
- **Input Fields / Key Information Needed:**
  - **Date:** (Selectable date)
  - **Activity/Task Description:** (Text area for detailed description)
  - **Project Associated:** (Dropdown/selection of assigned projects)
  - **Start Time:** (Time picker)
  - **End Time:** (Time picker)
  - **Total Hours:** (Auto-calculated or manually entered)
  - **Add Row Button:** (To add multiple activities for a day)
  - **Submit Button:** (To submit the timesheet, often weekly or bi-weekly)

---

9. Leave Application: Digital leave request system with approval tracking

- **What the student sees/does:** Requests time off and sees the status of their requests.
- **Input Fields / Key Information Needed:**
  - **Leave Type:** (Dropdown: e.g., "Sick Leave," "Casual Leave," "Unpaid Leave")
  - **Start Date:** (Date picker)
  - **End Date:** (Date picker)
  - **Reason for Leave:** (Text area for explanation)
  - **Contact Person during Leave:** (Name, Contact Info - optional)
  - **Submit Button:** (To send the request)
  - **Approval Status:** (System-updated: "Pending," "Approved," "Rejected")
  - **Approver Comments:** (System-provided if applicable)

7. Performance & Analytics

Technologies: Chart.js/D3.js for data visualization (This is an output/display technology, not an input requirement itself,

Performance Dashboard: Comprehensive performance metrics

- **What it needs:** A summary of all key performance indicators.
- **Input Fields (Data Sources):**
  - `student_id`: Unique identifier for the student.
  - `course_id`: Identifier for the course.
  - `quiz_score`: Individual quiz scores.
  - `assignment_score`: Individual assignment scores.
  - `task_completion_status`: (e.g., 'completed', 'pending', 'overdue').
  - `time_spent_on_course`: Duration in minutes/hours.
  - `attendance_status`: (e.g., 'present', 'absent', 'late').

- `internship_phase`: Current phase of the internship (e.g., 'Phase 1', 'Phase 2').
- `feedback_rating`: Numerical or categorical feedback (e.g., 1-5 stars, 'Excellent', 'Good').

Skill Assessment: Visual skill level indicators and improvement suggestions

- **What it needs:** Data representing skill proficiency and ways to suggest improvements.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `skill_id`: Unique identifier for a specific skill (e.g., 'Python Programming', 'Project Management').
  - `skill_level_score`: Numerical score (e.g., 1-100) or categorical (e.g., 'Beginner', 'Intermediate', 'Advanced') for each skill.
  - `assessment_date`: Date of the skill assessment.
  - `assessment_type`: (e.g., 'Self-assessment', 'Instructor-assessed', 'Quiz-based').
  - `quiz_question_category`: For quizzes, linking questions to specific skills to infer proficiency.
  - `suggestion_id`: For pre-defined improvement suggestions linked to skill levels.
  - `suggestion_text`: Text for improvement suggestions (can be stored in a separate table and linked).

• Learning Analytics: Time spent on courses, completion rates, quiz scores

- **What it needs:** Detailed tracking of learning activities.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `course_id`:
  - `module_id`: Identifier for modules within a course.
  - `lesson_id`: Identifier for lessons within a module.
  - `time_start`: Timestamp when a student starts a learning session/module/lesson.
  - `time_end`: Timestamp when a student ends a learning session/module/lesson.
  - `quiz_id`: Identifier for a specific quiz.
  - `quiz_score`: Score obtained on a quiz.
  - `quiz_max_score`: Maximum possible score for a quiz.
  - `course_status`: (e.g., 'In Progress', 'Completed', 'Not Started').
  - `module_completion_status`: (e.g., 'Completed', 'In Progress').
  - `lesson_completion_status`: (e.g., 'Completed', 'In Progress').

• Task Performance: Task completion rates, average scores, feedback trends

- **What it needs:** Data related to specific tasks or assignments.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `task_id`: Unique identifier for a specific task/assignment.
  - `task_due_date`: When the task is due.
  - `task_submission_date`: When the task was submitted.
  - `task_status`: (e.g., 'Submitted', 'Pending Review', 'Completed', 'Late Submission').
  - `task_score`: Score obtained on the task.
  - `task_max_score`: Maximum possible score for the task.
  - `feedback_id`: Identifier for specific feedback given.
  - `feedback_text`: Actual feedback comments.
  - `feedback_category`: (e.g., 'Clarity', 'Accuracy', 'Timeliness').
  - `feedback_timestamp`: When feedback was given.

• Attendance Reports: Visual attendance tracking with calendar view

- **What it needs:** Records of student presence.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `session_date`: Date of the attendance session (e.g., class, workshop).
  - `session_type`: (e.g., 'Class', 'Workshop', 'Meeting').
  - `attendance_status`: (e.g., 'Present', 'Absent', 'Late', 'Excused').
  - `check_in_time`: (Optional) Timestamp for when the student checked in.
  - `check_out_time`: (Optional) Timestamp for when the student checked out.

• Progress Reports: Detailed reports for different internship phases

- **What it needs:** Structured data to show progress across different stages.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `internship_id`: Unique identifier for the internship.
  - `internship_phase`: Current or completed phase (e.g., 'Phase 1: Onboarding', 'Phase 2: Project Work').
  - `phase_start_date`: Start date of the internship phase.
  - `phase_end_date`: End date of the internship phase.
  - `phase_completion_status`: (e.g., 'Completed', 'In Progress', 'Not Started').
  - `mentor_feedback_phase`: Overall feedback for the phase from the mentor.
  - `project_milestone_completion`: (e.g., 'Milestone 1 Achieved', 'Milestone 2 Pending').
  - `phase_specific_skill_assessment`: Skill scores relevant to that phase.

• Comparative Analysis: Performance comparison with peer groups

- **What it needs:** Data from multiple students within a defined group.
- **Input Fields (Data Sources):** (This feature primarily relies on aggregating and comparing the *existing* data points from other students, rather than entirely new input fields.)
  - `student_id`: (For the target student).
  - `peer_group_id`: Identifier for a group of students (e.g., 'Cohort 2025-A', 'Batch 1').
  - *All relevant performance metrics listed above (quiz scores, task scores, time spent, skill levels, etc.)* for **all students within the `peer_group_id`**.

• Goal Setting: Personal goal setting and tracking system

- **What it needs:** Student-defined goals and their progress towards them.
- **Input Fields (Data Sources):**
  - `student_id`:
  - `goal_id`: Unique identifier for a specific goal.
  - `goal_description`: Text describing the student's goal (e.g., 'Improve Python skills by 20%', 'Complete 3 machine learning projects').
  - `goal_target_value`: (e.g., '85%' for a skill score, '3' for projects).
  - `goal_current_value`: Current progress towards the goal.
  - `goal_start_date`: When the goal was set.
  - `goal_target_date`: When the student aims to achieve the goal.
  - `goal_status`: (e.g., 'Active', 'Achieved', 'Abandoned', 'In Progress').
  - `goal_category`: (e.g., 'Skill Development', 'Course Completion', 'Project Work').
  - `last_updated`: Timestamp of the last update to the goal's progress.

8.Communication & Support

Core Technologies:

- **Real-time Communication:** Socket.io
- **Notifications:** Browser/Mobile Notification APIs (e.g., Push API, Web Notifications API)

. Analyze Each Feature and Identify Input Fields:

- **Mentor Communication (Direct Messaging):**
  - Purpose: Facilitate private text-based communication between students and their assigned mentors.

- Input Fields:
  - `Sender ID (Student ID)`: Who is sending the message.
  - `Recipient ID (Mentor ID)`: Who is receiving the message.
  - `Message Content (Text Area)`: The actual message.
  - `Timestamp (Automatic)`: When the message was sent.
  - `Read Status (Boolean)`: Whether the message has been read by the recipient.

Video Calling (One-on-one sessions):

- Purpose: Enable real-time audio/video interaction between students and mentors.
- Input Fields:
  - `Caller ID (Student ID)`: Who initiated the call.
  - `Callee ID (Mentor ID)`: Who is being called.
  - `Call Session ID (Unique ID)`: Identifier for the ongoing call.
  - `Call Start Time (Automatic)`: When the call began.
  - `Call End Time (Automatic)`: When the call ended.
  - `Call Status (Enum: "Initiated", "Active", "Ended", "Declined")`: Current state of the call.
  - *(Optional but good to consider):* `Meeting Link/Room ID (Generated)`: For session initiation.

Discussion Forums (Course-Specific)

- **What it does:** Areas where students can post questions and discuss topics related to a specific course. Like an online message board for each class.
- **Technologies Used:** Standard database (e.g., SQL/NoSQL) for storing posts, comments; potentially `Socket.io` for real-time updates on new posts/comments if desired.
- **Simple Input Fields Needed (when creating a forum, thread, or post):**
  - **For a new Forum/Topic:**
    - `Course ID`: (Which course this forum belongs to)
    - `Forum Title/Topic`: (Name of the discussion topic, e.g., "Module 1 Questions")
    - `Forum Description`: (Brief explanation of what the forum is for)
  - **For a new Post/Question:**
    - `Forum ID`: (Which discussion forum this post is in)
    - `Author ID`: (Who posted it - automatically captured)
    - `Post Title`: (Subject of the post/question)
    - `Post Content`: (The main text of the post)
    - `Timestamp`: (When it was posted - automatically generated)
  - **For a Reply/Comment:**
    - `Post ID`: (Which post this is a reply to)
    - `Author ID`: (Who replied - automatically captured)
    - `Comment Content`: (The text of the reply)
    - `Timestamp`: (When it was replied - automatically generated)

Help Desk (Support Ticket System)

- **What it does:** Students can submit formal requests for technical help or other issues, and track their progress.
- **Technologies Used:** Standard database for tickets, potentially notification APIs for status updates.
- **Simple Input Fields Needed (when creating a new ticket):**
  - `Student ID`: (Who is submitting the ticket - automatically captured)
  - `Issue Type`: (e.g., "Technical Issue," "Account Problem," "Course Access") - *dropdown menu*
  - `Subject`: (Brief title for the issue, e.g., "Cannot log in")
  - `Description`: (Detailed explanation of the problem)
  - `Priority Level`: (e.g., Low, Medium, High - *dropdown menu*)
  - `Attachment(s)`: (Optional file upload for screenshots/documents)
  - `Timestamp Submitted`: (Automatically generated)
  - `Status`: (e.g., "New," "In Progress," "Resolved," "Closed" - automatically updated by admin)
  - `Assigned Agent ID`: (Which admin/support staff is handling it - assigned by admin)

FAQ Section (Searchable Knowledge Base)

- **What it does:** A library of common questions and answers that students can search through to find solutions themselves.
- **Technologies Used:** Standard database for storing FAQs, search indexing (e.g., Elasticsearch, or simple database text search).
- **Simple Input Fields Needed (when adding a new FAQ by admin):**
  - `Question`: (The common question, e.g., "How do I reset my password?")
  - `Answer`: (The detailed solution)
  - `Category/Topic`: (e.g., "Account," "Billing," "Course Navigation" - for organization)
  - `Keywords/Tags`: (Words that help with searching, e.g., "password," "login," "reset")
  - `Last Updated Date`: (Automatically generated/updated)

---

6. Live Chat (Real-time Support with Admin Staff)

- **What it does:** Students can instantly chat with a support person for quick help.
- **Technologies Used:** `Socket.io` (for real-time chat), Notification APIs.
- **Simple Input Fields Needed (when starting a chat session):**
  - `Student ID`: (Who is starting the chat - automatically captured)
  - `Admin ID`: (Which admin is chatting with the student - assigned by the system/admin)
  - `Chat Session ID`: (Unique ID for the conversation - automatically generated)
  - `Message Content`: (The actual text of each message in the chat)

- `Timestamp`: (When each message was sent - automatically generated)
- `Chat Status`: (e.g., "Active," "Ended," "Waiting" - automatically updated)

---

7. Announcement Board (Important Announcements & Updates)

- **What it does:** A central place where administrators can post important news, updates, or deadlines for all students to see.
- **Technologies Used:** Standard database for storing announcements, Notification APIs (to alert students about new announcements).
- **Simple Input Fields Needed (when creating a new announcement by admin):**
  - `Title`: (Subject of the announcement, e.g., "Platform Maintenance")
  - `Content`: (The full text of the announcement)
  - `Author ID`: (Who posted the announcement - admin ID)
  - `Publication Date`: (When it should appear - automatically generated, or set by admin)
  - `Expiry Date`: (Optional - when the announcement should disappear)
  - `Target Audience`: (e.g., "All Students," "Specific Course Students") - *dropdown/checkboxes*
  - `Pinned Status`: (Yes/No - if it should stay at the top)

9.Mobile Responsiveness & Accessibility

Technologies: Responsive CSS, PWA capabilities

Mobile-First Design

- **Description:** The dashboard will be designed and developed with mobile devices as the primary target, ensuring optimal layout, performance, and user experience on smaller screens before scaling up to tablets and desktops.
- **Why it's important:** Ensures core functionality is robust and user-friendly on the most common access point for many students, avoiding a "shrunk-down desktop" feel.
- **Input Fields / Necessary Information (for development/configuration):**
  - **Viewport Breakpoints:** Define specific screen widths (e.g., small mobile, large mobile, tablet, small desktop, large desktop) where the layout will adapt. (e.g., `$mobile-breakpoint: 768px;`)
  - **Flexible Grid System:** Specify the chosen CSS framework or custom grid system (e.g., Flexbox, CSS Grid) that supports dynamic content resizing.
  - **Image Optimization Strategy:** How images will be scaled or delivered (e.g., `srcset`, `picture` element, server-side resizing) to reduce load times on mobile.
  - **Font Sizing & Line Heights:** Define a responsive typographic scale that adjusts based on screen size for readability.

---

2. Offline Capability (PWA)

- **Description:** Students will be able to download course materials (videos, notes, quizzes) to access them even without an active internet connection. This relies heavily on Progressive Web App (PWA) features.
- **Why it's important:** Crucial for students in areas with unreliable internet or for those who wish to study on the go without data consumption.
- **Input Fields / Necessary Information:**
    - **Cache Strategy:** Define which types of assets (e.g., static files, course content) will be cached and how often the cache will be updated (e.g., "cache first, then network," "network first, then cache").
    - **Storage Limits:** Consider the maximum amount of data a user can download for offline viewing. This might be a user preference or a system-defined limit. (e.g., `$max-offline-storage-mb: 500;`)
    - **Content Selection for Offline:** How users will select specific courses or modules for offline download. This implies UI elements (buttons, checkboxes) and the backend logic to retrieve and store content.
    - **Sync Logic:** How downloaded content will be synchronized when the user regains internet access (e.g., progress tracking, updated course materials).

---

3. Push Notifications

- **Description:** The dashboard will leverage mobile push notifications to alert students about important updates such as new course content, assignment deadlines, announcements, or personalized recommendations.
- **Why it's important:** Keeps students engaged and informed, improving retention and timely completion of tasks.
- **Input Fields / Necessary Information:**
    - **Notification Types:** Define the categories of notifications (e.g., "New Course Added," "Assignment Due," "Announcement," "Study Reminder").
    - **Payload Structure:** What information will be included in each notification (e.g., title, body, link to relevant content, image).
    - **Opt-in/Opt-out Mechanism:** How users will grant/revoke permission for notifications. This requires a clear UI element in settings.
    - **Notification Triggers:** Define the events or schedules that will trigger specific notifications (e.g., "7 days before assignment deadline," "immediately upon new announcement").
    - **Backend Integration:** Details of the push notification service (e.g., Firebase Cloud Messaging, OneSignal) and its API keys.

---

4. Touch Optimization

- **Description:** All interactive elements (buttons, links, forms, sliders) will be designed with touch interaction in mind, featuring appropriate sizing, spacing, and feedback for mobile users.

- **Why it's important:** Ensures comfortable and error-free interaction on touchscreens, preventing accidental taps and improving user satisfaction.
- **Input Fields / Necessary Information:**
    - **Minimum Touch Target Size:** Define the minimum recommended size for tappable elements (e.g., `$min-touch-target-px: 48px;`).
    - **Spacing Between Elements:** How much padding or margin will be applied around interactive elements to prevent accidental taps on adjacent items.
    - **Hover States (Fallback):** How elements will visually respond on non-touch devices (hover) versus touch devices (active state).
    - **Gesture Recognition:** If any custom gestures are planned (e.g., swipe to navigate), define their behavior and expected input.

---

5. Fast Loading

- **Description:** The dashboard will be optimized for quick loading times, especially on low bandwidth connections, through efficient code, optimized assets, and effective caching strategies.
- **Why it's important:** Reduces user frustration, improves engagement, and is crucial for students with limited data plans or slow internet.
- **Input Fields / Necessary Information:**
    - **Code Splitting Strategy:** How the application's JavaScript and CSS will be broken down into smaller, on-demand chunks to load only what's necessary.
    - **Asset Compression:** Specify the compression techniques used for images (e.g., WebP, JPEG optimization), CSS (minification), and JavaScript (minification, tree-shaking).
    - **Lazy Loading:** Which content (e.g., images, videos below the fold) will be loaded only when it's about to enter the viewport.
    - **Content Delivery Network (CDN) Usage:** If a CDN will be used for static assets to distribute content closer to users globally.
    - **Server-Side Rendering (SSR) / Static Site Generation (SSG):** If applicable, how these techniques will be used to deliver initial HTML quickly.
    - **Performance Monitoring Tools:** Which tools will be used to measure and track loading performance (e.g., Lighthouse scores, Web Vitals).

---

6. Accessibility Standards (WCAG 2.1 Compliance)

- **Description:** The dashboard will adhere to Web Content Accessibility Guidelines (WCAG) 2.1 at a specified conformance level (e.g., AA) to ensure it is usable by individuals with disabilities.
- **Why it's important:** Ensures inclusivity and legal compliance, making the platform usable by the widest possible audience, including those with visual, auditory, motor, or cognitive impairments.
- **Input Fields / Necessary Information:**
    - **Conformance Level:** Specify the target WCAG 2.1 conformance level (A, AA, or AAA). AA is generally recommended.

- **Semantic HTML:** Requirement for using appropriate HTML5 elements (e.g., `button`, `nav`, `main`, `form` with `label`) to convey meaning to assistive technologies.
- **ARIA Attributes:** Guidelines for using Accessible Rich Internet Applications (ARIA) attributes when native HTML isn't sufficient (e.g., `aria-label`, `aria-describedby`, `aria-haspopup`).
- **Keyboard Navigation:** All interactive elements must be reachable and operable using only a keyboard, with clear focus indicators.
- **Color Contrast Ratios:** Minimum contrast ratios for text and interactive elements against their backgrounds (e.g., 4.5:1 for normal text, 3:1 for large text).
- **Alternative Text for Images:** Requirement for descriptive `alt` attributes on all meaningful images.
- **Form Labeling:** All input fields must have explicitly associated labels. (e.g., `<label for="username">Username:</label><input type="text" id="username">`)
- **Error Handling (Accessible):** How form validation errors will be communicated to assistive technologies.
- **Transcripts & Captions:** For audio/video content, requirement for providing transcripts and closed captions.
- **Accessibility Testing Tools:** Which tools will be used for testing (e.g., Axe-core, WAVE, screen readers).

10.Data Security & Privacy

This section deals with crucial security and privacy aspects. Input fields here would primarily be for configuration, policy definitions, and system settings.

- **Technologies:** SSL/TLS encryption, JWT authentication
  - **Input Fields/Considerations:**
    - `SSL/TLS Certificate Upload Field:` For uploading SSL/TLS certificates (e.g., `.pem`, `.crt` files).
    - `Certificate Password Field:` If the private key is password-protected.
    - `JWT Secret Key Input:` A secure field for entering the JWT secret key (for token signing).
    - `JWT Expiration Time (minutes/hours/days):` Numerical input for token validity.
    - `JWT Algorithm Selection:` Dropdown (e.g., HS256, RS256).

- **Secure Login: Multi-factor authentication options**
  - **Input Fields/Considerations (for configuration):**
    - `Enable/Disable MFA (Checkbox):` System-wide setting.
    - `MFA Type Selection (Dropdown/Checkboxes):` (e.g., OTP via Email, OTP via SMS, Authenticator App).

        - `SMS Gateway API Key/Credentials`: If SMS MFA is enabled.
        - `Email SMTP Settings`: For email OTP.
        - `MFA Enrollment Status (User Profile)`: For individual users to manage their MFA.

- **Data Encryption: End-to-end encryption for sensitive data**
    - **Input Fields/Considerations:**
        - `Encryption Key Management Strategy (Text Area/Policy Document Link)`: Description of how encryption keys are generated, stored, and rotated. This isn't an "input field" in the traditional sense, but a crucial piece of documentation/configuration.
        - `Data Fields to Encrypt (Configuration List)`: A list or selection of specific database fields identified as sensitive (e.g., `user.password`, `payment_info.card_number`).

- **Privacy Controls: User data management and deletion options**
    - **Input Fields/Considerations (for user interface):**
        - `Request Data Export Button`: On the user's privacy settings page.
        - `Delete Account Button`: On the user's privacy settings page (often with confirmation).
        - `Data Retention Policy (Display Only)`: Text area to display the policy to the user.
        - `Consent Management Checkboxes`: For specific data usage (e.g., marketing emails, analytics tracking).

- **Session Management: Secure session handling with timeout**
    - **Input Fields/Considerations (for configuration):**
        - `Session Timeout Duration (minutes)`: Numerical input.
        - `Remember Me Option (Checkbox)`: For user-controlled longer session duration (with warnings).
        - `Automatic Logout after Inactivity (Checkbox)`: To enforce logout after timeout.

- **GDPR Compliance: Privacy policy acceptance and data portability**
    - **Input Fields/Considerations:**
        - `Privacy Policy URL/Content (Text Area)`: For displaying and updating the privacy policy.
        - `Terms of Service URL/Content (Text Area)`: For displaying and updating the terms of service.
        - `"I Accept Privacy Policy" Checkbox`: Mandatory for new user registration.

        - `"Request Data Export" Functionality:` As mentioned in "Privacy Controls."

- **Audit Logs: User activity logging for security purposes**
    - **Input Fields/Considerations (for internal system configuration):**
        - `Log Level Configuration (Dropdown):` (e.g., Info, Warning, Error, Debug).
        - `Retention Period for Audit Logs (days/weeks/months):` Numerical input.
        - `Events to Log (Checklist/Configuration):` (e.g., Login attempts, Password changes, Data deletions, Course enrollments).
        - `Log Storage Location:` (e.g., Database, File System, Cloud Logging Service).

11. Integration Features

This section focuses on connecting with external services. Input fields here will primarily be for API keys, credentials, and configuration settings for these integrations.

- **Technologies:** RESTful APIs, third-party integrations
    - **Input Fields/Considerations:** These are general considerations for all integrations:
        - `API Key/Token Field:` For secure input of API keys.
        - `Client ID/Client Secret Fields:` For OAuth-based integrations.
        - `Webhook URL (if applicable):` For receiving data from external services.
        - `Enable/Disable Integration (Checkbox):` For toggling individual integrations.

- **Calendar Integration: Sync with Google Calendar/Outlook**
    - **Input Fields/Considerations (for user interface):**
        - `"Connect Google Calendar" Button:` Triggers OAuth flow.
        - `"Connect Outlook Calendar" Button:` Triggers OAuth flow.
        - `Calendar Selection (Dropdown):` If a user has multiple calendars, allow selection.
        - `Sync Preferences (Checkboxes):` (e.g., "Sync all course deadlines," "Sync only marked events").

- **Social Media: Share achievements on social platforms**
    - **Input Fields/Considerations (for user interface and admin config):**

        - `"Share to [Platform]" Buttons:` (e.g., LinkedIn, Twitter, Facebook) on achievement pages.
        - `Pre-filled Share Message Template (Admin Config):` Editable text area for default share messages.
        - `Social Media API Keys (Admin Config):` (e.g., LinkedIn Client ID/Secret, Twitter Consumer Key/Secret).

- **Email Integration: Automated email notifications and updates**
    - **Input Fields/Considerations (for admin configuration):**
        - `SMTP Host, Port, Username, Password, Encryption Type (SSL/TLS):` For sending emails.
        - `"From" Email Address:` The sender's email address.
        - `Email Templates (Text Area/HTML Editor):` For various notifications (e.g., Welcome, Password Reset, Course Completion).
        - `Notification Preferences (User Profile):` Checkboxes for users to select which email notifications they want to receive.

- **Export Functionality: Export certificates, reports in PDF format**
    - **Input Fields/Considerations (for user interface):**
        - `"Export Certificate as PDF" Button:` On certificate view pages.
        - `"Export Report as PDF" Button:` On report view pages.
        - `Report Type Selection (Dropdown):` If multiple report types are available.
        - `Date Range Pickers:` For specifying data range for reports.

- **API Access: Personal API for third-party integrations**
    - **Input Fields/Considerations (for user interface):**
        - `"Generate API Key" Button:` On a dedicated "API Access" section in user settings.
        - `API Key Display Field (Read-only, with copy option):` Once generated.
        - `"Revoke API Key" Button:` For security.
        - `API Documentation Link:` Link to the personal API documentation.

12. Technical Implementation Details

This section describes the technology stack and architectural choices. While not directly "input fields" for users or administrators, these are critical considerations that influence the design of input forms, the validation rules, and the overall user experience.

- **Frontend Stack:** React.js 18+, Tailwind CSS, React Router, Axios, Chart.js

- **Input Fields/Considerations (for developer/designer):**
    - **React.js 18+ with hooks and context API:** Implies component-based design. Each "input field" will likely correspond to a React component (e.g., `TextInput`, `SelectField`, `Checkbox`).
    - **Tailwind CSS for styling:** Impacts the design system and how forms look. No direct "input fields" related to this from a user perspective, but configuration for themes/colors might exist at a global level for designers.
    - **React Router for navigation:** Defines how URLs map to different views, which affects the design of navigation menus and links to forms.
    - **Axios for API calls:** Defines how form submissions will communicate with the backend. Input fields will have associated validation rules and submission logic.
    - **Chart.js for data visualization:** Input fields would relate to *what* data is displayed in charts (e.g., "Select Course Performance Metric," "Time Period for Chart").

- **Backend Integration:** RESTful API consumption, Real-time WebSocket connections, File upload with progress tracking, Authentication token management, Error handling and retry mechanisms
    - **Input Fields/Considerations:**
    - **RESTful API consumption:** Every form submission will be an API call. Input fields will map directly to JSON payloads sent to the backend.
    - **Real-time WebSocket connections:** Not directly related to "input fields" but influences how dynamic updates are received (e.g., real-time chat, live notifications).

        - **File upload with progress tracking:**
            - `File Input Field (`<input type="file">`)`: For uploading documents (e.g., project submissions, profile pictures).
            - `Progress Bar/Indicator`: UI element to show upload status.
            - `File Size Limit (Admin Config)`: Numerical input for maximum file size.
            - `Allowed File Types (Admin Config)`: List of permissible file extensions.

        - **Authentication token management:** This is mostly internal, but impacts how users log in and how their session is maintained (e.g., `Username/Email` and `Password` input fields for login).

        - **Error handling and retry mechanisms:** These influence the feedback provided to the user when an input is invalid or a submission fails (e.g., "Invalid email format," "Server error, please try again").

- **Performance Optimization:** Lazy loading, Image optimization, Caching strategies, Bundle optimization
    - **Input Fields/Considerations:** These are primarily internal development strategies and don't directly correspond to user-facing input fields, but they impact the *performance* of forms and overall UI responsiveness.
        - **Image Optimization:** If users upload images, there might be options for quality/compression during upload, or automatic server-side processing.
        - **Caching:** Impacts how quickly data appears in forms or reports.